

SDF PROJECT

Thumma Tejaswil

Introduction

The objective of this project is to build a library in Java to perform $+$, $-$, $*$, $/$ operations on two very long numbers which causes overflows in general java usage.

The project contains a main class file `MyInfArith.java` and two files `AInteger.java` and `AFloat.java`.

- Project/
 - `arbitraryarithmetic/`
 - * `MyInfArith.java`
 - * `AInteger.java`
 - * `AFloat.java`
 - `build.xml`
 - `compile.py`
 - `Script.py`
 - `defaulttestcase.txt`
 - `Read me.md`

Overview of Project

The project consists of three main classes: `AInteger` and `AFloat`, which handle arbitrary-precision arithmetic operations on integers and floating-point numbers respectively

`AInteger`

1 `AInteger` Class

The `AInteger` class provides support for arbitrary-precision integer arithmetic using string-based internal representation. It implements core arithmetic operations without relying on Java's built-in numeric types.

1.1 Class Overview

- `AInteger()` – Default constructor initializing value to 0.
- `AInteger(String s)` – Constructs an instance from a string.
- `parse(String s)` – Static parser method.
- `toString()` – Returns string representation.

1.2 Arithmetic Operations

- **Addition:**
 - Handled via `add(AInteger other)`.
 - Internally calls `add_these_both_strings()` to perform digit-by-digit addition.
 - Manages signed inputs: $a + b$, $-a + b$, etc.
- **Subtraction:**
 - Handled via `subtract(AInteger other)`.
 - Calls `subtract_these_both_strings()` for positive numbers.
 - Sign logic allows support for expressions like $a - (-b)$.
- **Multiplication:**
 - Performed via `multiply(AInteger other)`.
 - Calls `multiply_these_both_strings()` to simulate long multiplication.
- **Division:**
 - Performed via `divide(AInteger other)`.
 - Uses `divide_these_both_strings()` to implement long division.
 - Handles divide-by-zero via exception.

1.3 Helper Functions

- `removing_leading_zeros(String s)`: Removes redundant leading zeros.
- `isSmaller(String a, String b)`: Compares two strings as numeric values.
- `stringToIntegerArray(String s)` and `arrayToString(int[] a)`: Used for conversions between numeric strings and arrays.

AFloat

Overview

The **AFloat** class implements arbitrary-precision floating-point arithmetic using string manipulation. It mimics high-precision math operations by breaking down floats into their integer and decimal components and using helper methods from the **AInteger** class.

1.3.1 Fields

String *float_value*;

1.3.2 Constructor

`AFloat();` // default, "0.0" `AFloat(String s);` // from string `AFloat(AFloat other);` // copy constructor

1.3.3 Addition

Addition works as:

$$\text{If signs match: } a.b + c.d = (a + c).(b + d)$$

If decimal addition overflows:

$$\text{carry from } b + d \rightarrow a + c + 1$$

Subtraction

Align decimals:

$$x = a.b, \quad y = c.d \rightarrow \text{convert to integers: } ab - cd$$

Insert decimal after subtraction, restore signs.

Multiplication

$$x = a.bc, \quad y = d.ef \rightarrow x \cdot y = (abc \cdot def) / 10^{(n_1 + n_2)}$$

Where n_1, n_2 are number of decimal digits in x, y .

Division

$$x = a.bc, \quad y = d.ef$$

Multiply numerator by 10^{30} to preserve 30 digits of precision:

$$\frac{x \cdot 10^{30}}{y} = z \Rightarrow \text{Insert decimal in } z$$

Precision

All operations maintain up to 30 decimal digits after the decimal point.

MyInfArith

The class `MyInfArith` acts as an interface between the user and the arithmetic classes, using methods from `AInteger` and `AFloat` to perform the desired operations based on user input.

Class Implementation

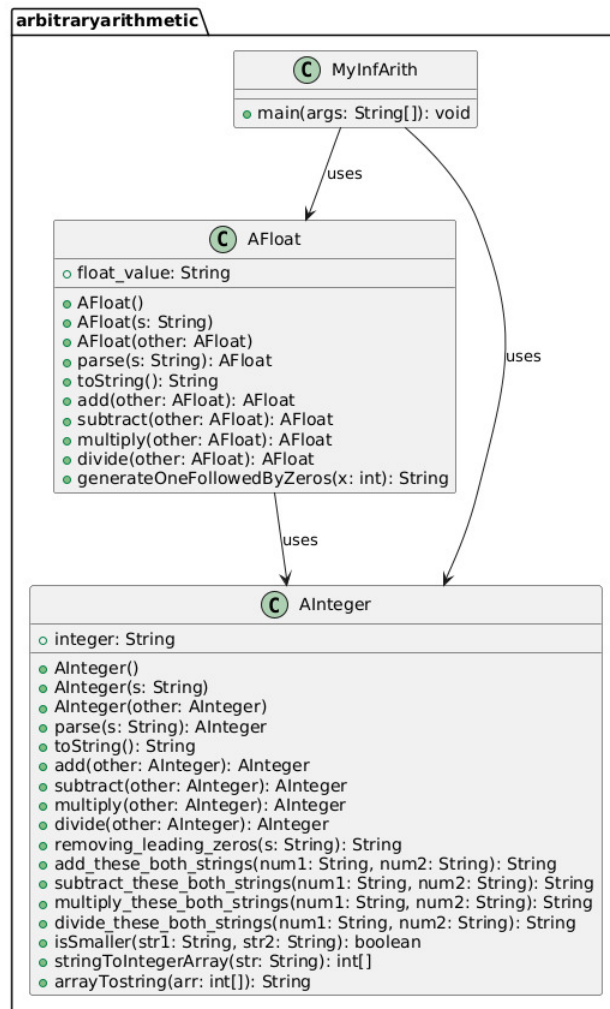


Figure 1: This is the caption of the image.

AIInteger

The main methods of this class are:

- `add()`: Performs addition on two arbitrary-length integers, handling both positive and negative values. It uses digit-wise summation with carry propagation. If the operands have opposite signs, the method delegates the operation to the `sub()` method to correctly manage sign and magnitude.

- **subtract()**: Performs subtraction on two integers of arbitrary size, correctly handling positive and negative cases. It uses digit-wise subtraction with borrow handling. If the operands have opposite signs, it internally calls the **add()** method to compute the result.
- **multiply()**: Performs multiplication of two arbitrary-precision integers. It uses digit-wise multiplication and handles the sign of the product appropriately, based on the signs of the operands.
- **AInteger.divide()** : The method implements arbitrary-precision integer division using a manual long division algorithm. It first processes the signs of the operands and handles special cases such as division by zero or a zero numerator. The division is carried out by repeatedly subtracting the divisor from portions of the dividend, constructing the quotient one digit at a time, much like traditional long division. The result's sign is determined based on the original signs of the operands: the result is negative if only one operand is negative and positive otherwise. Finally, the quotient is returned as a new **AInteger** object, with leading zeros removed for proper formatting.

AFloat

The main methods of this class are:

AFloat Addition The **add()** method in **AFloat** handles arbitrary-precision addition of floating-point numbers represented as strings. It separates the integer and fractional parts of both operands, equalizes their decimal lengths by padding with zeros, and adds them using the **AInteger** class. Carries from the fractional addition are handled by adjusting the integer sum accordingly, and the result is formatted with trailing zero trimming.

AFloat Subtraction The **subtract()** method performs floating-point subtraction by aligning both numbers in length and decimal placement. It manages sign changes and magnitude comparison to determine which number is greater, performs digit-by-digit subtraction (handling borrow), and ensures the result has no leading or unnecessary trailing zeros, preserving precision.

AFloat Multiplication The **multiply()** method removes decimal points from both operands and counts the number of decimal digits. It multiplies the resulting integers using the **AInteger** class and re-inserts the decimal point in the result based on the total number of decimal digits. It ensures the final result has up to 30 decimal places of precision.

AFloat Division The `divide()` method first removes decimal points and normalizes the operands by adding zeros for precision. It performs integer division via the `AInteger.divide()` method and re-inserts the decimal point based on the adjusted scale. The result is formatted to show up to 30 decimal digits, with appropriate handling for signs and division by zero.

Usage of Library

Command Line Interface

The project provides a command-line interface through the `build.xml` and `compile.py` and `script.py`. This class parses the input arguments and invokes the appropriate methods in either the `AInteger` or `AFloat` classes.

Usage Pattern:

```
python3 Script.py <int/float> <add/sub/mult/div> <number1> <number2>
```

Examples:

- `python3 Script.py i int add 164884844 -6846464646666`
Output: The Answer is -6681579801822
- `python3 Script.py float div 444634341544.13134 65633435434`
Output: The Answer is 6.77450964746846105188137575739381797
- `java python3 Script.py float sub 5.9585 -2.2565`
Output: The Answer is 8.2150

Script.py

The Python script `Script.py` automates the compilation and execution process. It uses Apache Ant to compile the Java files and then executes the program with the provided input arguments.

build.xml

The `build.xml` file is the Ant build script that defines multiple targets for building and managing the project.

- **clean:** Deletes all compiled files from the `build` directory.
- **compile:** Compiles all source files and outputs the class files into the `build` directory.

Conclusion

This arbitrary-precision arithmetic library provides a better solution for performing large number arithmetic operations in Java. By using string manipulation to handle large numbers, the library can support integers and floating-point numbers of arbitrary size.